



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР



GNU Bourne-Again Shell (Bash) – это мощная рабочая среда и механизм написания сценариев. Компетентный специалист по безопасности должен уметь использовать сценарии Bash для оптимизации и автоматизации задач и процедур Linux. В этом модуле мы познакомимся со сценариями Bash и рассмотрим несколько практических сценариев.

Сценарий Bash - это текстовый файл, содержащий ряд команд, которые выполняются так, как если бы они были набраны в терминале. Как правило, сценарии Bash имеют расширение `.sh`, начинаются с `#!/bin/bash` и должны иметь разрешения на выполнение. Давайте начнем с простого Bash-скрипта "Hello World":

```
kali@kali:~$ cat > ./hello-world.sh
```

```
#!/bin/bash
```

```
# Hello World
```

```
echo "Hello World!"
```

Этот сценарий состоит из нескольких компонентов, о которых стоит рассказать:

Строка 1: **#!** широко известна как shebang и игнорируется интерпретатором Bash. Вторая часть, **bin/bash**, является абсолютным путем к интерпретатору, который используется для запуска скрипта.

Строка 2: **#** используется для добавления комментария, поэтому весь текст, следующий за ним, игнорируется.

Строка 3: **echo "Hello World"** использует командную утилиту **echo** Linux для печати заданной строки в терминале.

Далее, давайте сделаем скрипт исполняемым и запустим его:

```
kali@kali:~$ chmod +x hello-world.sh
```

```
kali@kali:~$ ./hello-world.sh
```

```
Hello World!
```

Команда `chmod` вместе с опцией **+x** используется для того, чтобы сделать сценарий исполняемым, а `./helloworld.sh` используется для фактического запуска. Обозначение `./` может показаться непонятным, но это просто обозначение пути, указывающее, что этот скрипт находится в текущем каталоге. Всякий раз, когда мы вводим команду, Bash пытается найти ее в

ряде каталогов, хранящихся в переменной под названием PATH. Поскольку наш домашний каталог не включен в эту переменную, мы должны использовать относительный путь к нашему сценарию Bash, чтобы Bash нашел и запустил его.

Теперь, когда мы создали наш первый сценарий Bash, давайте изучим Bash более подробно.

Переменные

Переменные – это именованные места для временного хранения данных. Мы можем установить (или "объявить") переменную. Мы можем объявлять значения переменных несколькими способами. Самый простой способ - задать значение напрямую с помощью простого объявления ИМЯ=ЗНАЧЕНИЕ. Обратите внимание, что нет пробелов ни до, ни после знака =:

```
kali@kali:~$ var1=hacker
```

Объявление переменной бессмысленно, если мы не можем ссылаться на нее. Для этого мы предваряем переменную символом "\$". Всякий раз, когда Bash встречает этот синтаксис в команде, он заменяет имя переменной на ее значение перед выполнением:

```
kali@kali:~$ var1=hacker
```

```
kali@kali:~$ var2=hacker2
```

```
kali@kali:~$ echo $var1 $var2
```

```
hacker hacker2
```

Имена переменных могут быть прописными, строчными или сочетать оба варианта. Однако Bash чувствителен к регистру поэтому мы должны быть последовательны при объявлении переменных. Кроме того, хорошей практикой является использование описательных имен переменных, которые облегчают чтение и сопровождение наших скриптов. Имейте в виду, что Bash интерпретирует некоторые символы особым образом. Например, это объявление демонстрирует неправильное объявление переменной:

```
kali@kali:~$ var1=Hello World
```

```
bash: World: command not found
```

Это не совсем то, что мы ожидали. Чтобы исправить это, мы можем использовать либо одинарные кавычки ('), либо двойные кавычки ("), чтобы

заклучить наш текст. Однако Bash обрабатывает одинарные и двойные кавычки по-разному. При заключении в двойные кавычки, все символы воспринимаются, кроме "\$", "" и "\". Простой пример поможет прояснить это:

```
kali@kali:~$ var1='Hello World'
kali@kali:~$ echo $var1
Hello World
kali@kali:~$ var2="New $var1"
kali@kali:~$ echo $var2
New Hello World
```

В этом примере объявление приветствия, заключенное в одну кавычку, сохранило значение нашего текста и не интерпретирует пробел как разделитель команд. Мы также можем установить значение переменной как результат команды или программы. Этот способ позволяет нам взять вывод команды или программы (то, что обычно выводится на экран) и сохранить его в качестве значения переменной. Для этого нужно поместить имя переменной в круглые скобки "()", перед которыми должен стоять символ "\$":

```
kali@kali:~$ user=$(whoami)
kali@kali:~$ echo $user
kali
```

Мы присвоили вывод команды `whoami` переменной `user`. Затем мы отобрали ее значение.

Аргументы

Не все сценарии Bash требуют аргументов. Однако чрезвычайно важно понимать, как они интерпретируются Bash и как их использовать. Мы уже выполняли команды Linux с аргументами. Например, когда мы выполняем команду **ls -l /var/log**, **-l** и **/var/log** являются аргументами команды **ls**. Сценарии Bash ничем не отличаются, и мы можем использовать аргументы командной строки в наших сценариях:

```
kali@kali:~$ cat > ./arg.sh
#!/bin/bash
echo "$1 and $2"
kali@kali:~$ chmod +x ./arg.sh
kali@kali:~$ ./arg.sh hello world
hello and world
```

Мы создали простой сценарий Bash, установили для него исполняемые разрешения, а затем запустили его с двумя аргументами. Переменные **\$1** и **\$2** представляют первый и второй аргументы, переданные скрипту. Так же существуют несколько специальных переменных Bash, такие, как **\$HOSTNAME** (имя хоста), **\$RANDOM** (случайное число) и другие.

Чтение пользовательского ввода

Аргументы командной строки являются формой пользовательского ввода, но мы также можем перехватывать пользовательский ввод во время выполнения сценария с помощью команды **read**:

```
kali@kali:~$ cat > ./input.sh
#!/bin/bash
echo "You're a hacker: Y/N?"
read answer
echo "Your answer was $answer"
kali@kali:~$ chmod +x ./input.sh
kali@kali:~$ ./input.sh
You're a hacker: Y/N?
Y
```

Your answer was Y

Мы можем изменить поведение команды `read` с помощью различных опций командной строки. Две из наиболее часто используемых опций это **-p**, которая позволяет нам указать подсказку, и **-s**, которая делает пользовательский ввод "тихим":

```
kali@kali:~$ cat > ./input2.sh
#!/bin/bash
read -p 'Username: ' username
read -sp 'Password: ' password
echo "Thanks, your name and password: \"$username\" and \" $password"
kali@kali:~$ chmod +x ./input2.sh
kali@kali:~$ ./input2.sh
Username: kali
Password:
Thanks, your name and password: kali and Cyberden123
```

Знание Bash критически важно для работы с Linux, так как это основной командный интерпретатор в этой системе, обеспечивающий мощные возможности автоматизации и управления. Владея Bash, слушатель сможет эффективно выполнять рутинные задачи, создавать скрипты для автоматизации процессов, а также работать с файлами, сетями и сервисами на глубоком уровне. Для пентестеров Bash становится незаменимым инструментом, позволяющим быстро писать кастомные скрипты для сканирования уязвимостей, управления доступами и выполнения сложных команд в рамках тестирования безопасности, что значительно ускоряет процесс и повышает его эффективность.

Для более углубленного изучения, конечно же обращайтесь к встроенной справке `bash`:

```
man bash
```

Небольшой список книг, которые помогут вам овладеть более сложными техниками:

Командная строка Linux. Полное руководство. 2018. Питер. Шоттс У.

Идиомы bash. 2024. Питер. Олбинг К., Фоссен Д.